МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
”Сибирский государственный университет телекоммуникаций и информатики”

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

Отчет по практической работе № 2
по дисциплине
Параллельное программирование

Студент:
Группа ИА-331 *И.А. Иванов*

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ВЫПОЛНЕНИЕ РАБОТЫ 4
1.1 Постановка задачи и теоретическое обоснование 4
1.2 Реализация 4
1.2.1 Последовательный эталон 4
1.2.2 Ленточное разбиение по строкам 5
1.2.3 Ленточное разбиение по столбцам 5
1.2.4 Блочное разбиение $p \times q$ 6
1.2.5 Организация эксперимента 7
1.3 Экспериментальная установка 8
1.4 Результаты 8
1.5 Анализ результатов 10
1.6 Контрольные вопросы 10
1.6.1 Почему row-bands быстрее col-bands и blocks? 10
1.6.2 Зачем нужен `schedule(static)`? 11
1.6.3 Что даёт `omp_get_wtime` по сравнению с `clock`? 11
1.6.4 Почему при $T = 6, 8$ нет линейного ускорения? 11
1.6.5 Почему `t_ref` не зависит от T? 11

ЦЕЛИ И ЗАДАЧИ

Цель практической работы: освоить базовые механизмы OpenMP на примере классической задачи матрично-векторного произведения (GEMV) и оценить масштабируемость трёх схем декомпозиции данных при числе потоков $T = 1, 2, 4, 6, 8$.

Задачи работы:

1. Реализовать последовательный эталонный алгоритм `matvec_ref` для проверки корректности.
2. Реализовать ленточное разбиение матрицы по строкам с шириной ленты k.
3. Реализовать ленточное разбиение по столбцам с шириной ленты k.
4. Реализовать блочное разбиение с блоком $p \times q$.
5. Освоить директивы OpenMP: `parallel for`, `schedule`, `critical`, а также функции `omp_get_wtime`, `omp_set_num_threads`, `omp_get_max_threads`.
6. Проверить совпадение результатов всех трёх схем с эталоном с точностью 10^{-9}.
7. Провести экспериментальный анализ масштабируемости при $T = 1, 2, 4, 6, 8$, подобрав размеры задачи так, чтобы однопоточное время было порядка $O(1)$ секунды.
8. Построить графики ускорения и времени выполнения, оформить отчёт.

В качестве языка реализации используется C++ (компилятор g++, флаг `-fopenmp`); эксперименты проведены на машине с 8 логическими ядрами.

ВЫПОЛНЕНИЕ РАБОТЫ

1.1 Постановка задачи и теоретическое обоснование

Требуется вычислить вектор $c = A\,b$, где $A \in \$\mathbf{R}^{m\times n}$, $b \in \mathbf{R}^n$, элементы — тип `double`:

$$c_i = \sum_{j=0}^{n-1} a_{ij}\, b_j, \qquad i = 0, \ldots, m-1. \tag{1}$$

Число операций $W = 2mn$, объём читаемой памяти $\approx 8mn$ байт (матрица A плюс вектор b). Отсюда **арифметическая интенсивность**:

$$I = \frac{W}{\text{байт}} \approx \frac{2mn}{8mn} = 0.25 \text{ флоп/байт}. \tag{2}$$

Это очень низкое значение, поэтому задача GEMV является **memory-bound**: ускорение ограничено пропускной способностью памяти, а не числом ядер. Формально:

$$S(T) \;\le\; \min\left(T,\, \frac{B_{\text{peak}}}{B_{\text{seq}}}\right), \tag{3}$$

где B_{seq} — пропускная способность одного ядра, B_{peak} — всей системы. Этот факт объясняет поведение программы при больших T.

1.2 Реализация

1.2.1 Последовательный эталон

Для проверки корректности реализован однопоточный алгоритм `matvec_ref`, результат которого сравнивается с результатами трёх параллельных схем с точностью 10^{-9}.

```
static void matvec_ref(int m, int n, const double *A, const double *b,
    double *c) {
     for (int i = 0; i < m; ++i) {
         double s = 0.0;
         for (int j = 0; j < n; ++j) s += A[(size_t)i * n + j] * b[j];
         c[i] = s;
     }
}
```

Листинг 1.1 — Последовательный GEMV

1.2.2 Ленточное разбиение по строкам

Матрица делится на $\lceil m/k \rceil$ горизонтальных лент по k строк. Каждая лента даёт свой диапазон компонент c_i, гонок по записи нет. Директива `parallel for` с `schedule(static)` раздаёт ленты потокам.

```
static void matvec_row_bands(int m, int n, const double *A, const
    double *b,
                             double *c, int k) {
     #pragma omp parallel for schedule(static)
     for (int band = 0; band < (m + k - 1) / k; ++band) {
         int i0 = band * k;
         int i1 = i0 + k; if (i1 > m) i1 = m;
         for (int i = i0; i < i1; ++i) {
             double s = 0.0;
             for (int j = 0; j < n; ++j) s += A[(size_t)i * n + j] *
                 b[j];
             c[i] = s;
         }
     }
}
```

Листинг 1.2 — Ленты по строкам

1.2.3 Ленточное разбиение по столбцам

Матрица делится на $\lceil n/k \rceil$ вертикальных лент. Каждая лента вносит вклад во все c_i, поэтому возникает гонка по записи в c. Она устраняется локальным массивом `local[m]` на поток и финальным сведением через `critical`.

```
static void matvec_col_bands(int m, int n, const double *A, const
    double *b,
                             double *c, int k) {
     int nbands = (n + k - 1) / k;
     #pragma omp parallel
     {
         double *local = (double *)calloc(m, sizeof(double));
```

```
        #pragma omp for schedule(static)
        for (int band = 0; band < nbands; ++band) {
            int j0 = band * k;
            int j1 = j0 + k; if (j1 > n) j1 = n;
            for (int i = 0; i < m; ++i) {
                double s = 0.0;
                for (int j = j0; j < j1; ++j)
                    s += A[(size_t)i * n + j] * b[j];
                local[i] += s;
            }
        }
        #pragma omp critical
        {
            for (int i = 0; i < m; ++i) c[i] += local[i];
        }
        free(local);
    }
}
```

Листинг 1.3 — Ленты по столбцам

1.2.4 Блочное разбиение $p \times q$

Матрица делится на $m_b = \lceil m/p \rceil$ блочных строк и $n_b = \lceil n/q \rceil$ блочных столбцов. Параллельный цикл идёт по $m_b n_b$ блокам; используется тот же приём с локальным буфером и `critical`. При $p = q = 64$ блок занимает $64 \cdot 64 \cdot 8 = 32$ КБ, что помещается в кэш L1.

```
static void matvec_blocks(int m, int n, const double *A, const double
   *b,
                          double *c, int p, int q) {
    int mb = (m + p - 1) / p;
    int nb = (n + q - 1) / q;
    int total = mb * nb;
    #pragma omp parallel
    {
        double *local = (double *)calloc(m, sizeof(double));
        #pragma omp for schedule(static)
        for (int t = 0; t < total; ++t) {
            int bi = t / nb;
            int bj = t % nb;
            int i0 = bi * p, i1 = i0 + p; if (i1 > m) i1 = m;
            int j0 = bj * q, j1 = j0 + q; if (j1 > n) j1 = n;
```

```
            for (int i = i0; i < i1; ++i) {
                double s = 0.0;
                for (int j = j0; j < j1; ++j)
                    s += A[(size_t)i * n + j] * b[j];
                local[i] += s;
            }
        }
        #pragma omp critical
        {
            for (int i = 0; i < m; ++i) c[i] += local[i];
        }
        free(local);
    }
}
```

Листинг 1.4 — Блочное разбиение

1.2.5 Организация эксперимента

Программа принимает аргументы командной строки:

```
./lb2 <m> <n> <k> <p> <q>
```

Листинг 1.5 — Запуск

и последовательно прогоняет все три алгоритма для $T \in \{1, 2, 4, 6, 8\}$. Число потоков ограничивается сверху значением `omp_get_max_threads()`. Таймер — `omp_get_wtime()` (wall-clock). Результаты сохраняются в файл `results.csv` и дублируются в консоль в сгруппированном по T виде: сначала всё для $T = 1$, затем для $T = 2$ и т.д.

```
int max_t = omp_get_max_threads();
int thread_list[] = {1, 2, 4, 6, 8};
for (int idx = 0; idx < 5; ++idx) {
    int T = thread_list[idx];
    if (T > max_t) T = max_t;
    omp_set_num_threads(T);
    double t0 = omp_get_wtime();
    /* ...           ... */
    double t_alg = omp_get_wtime() - t0;
}
```

Листинг 1.6 — Управление потоками и таймер

1.3 Экспериментальная установка

– Компилятор: `g++ -O2 -fopenmp`.
– Размеры задачи: $m = n = 8000$ (матрица ≈ 512 МБ).
– Параметры разбиения: $k = 64$, $p = q = 64$.
– Число потоков: $T \in \{1, 2, 4, 6, 8\}$ (машина имеет 8 логических ядер).
– Таймер: `omp_get_wtime()`.

Размер $m = n = 8000$ подобран так, чтобы при $T = 1$ время выполнения было порядка одной секунды.

1.4 Результаты

Таблица 1 — Время выполнения (с) при $m = n = 8000$, $k = p = q = 64$

T	row-bands	col-bands	blocks	t_{ref}
1	0.951	1.048	1.102	0.942
2	0.501	0.579	0.602	0.941
4	0.270	0.331	0.340	0.943
6	0.201	0.268	0.279	0.941
8	0.183	0.252	0.261	0.940

Таблица 2 — Ускорение $S(T) = t(1)/t(T)$ и эффективность $E(T) = S(T)/T$

T	S_{row}	E_{row}	S_{col}	E_{col}	S_{blk}	E_{blk}
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.90	0.95	1.81	0.90	1.83	0.92
4	3.52	0.88	3.18	0.80	3.24	0.81
6	4.73	0.79	3.91	0.65	3.95	0.66
8	5.20	0.65	4.16	0.52	4.22	0.53

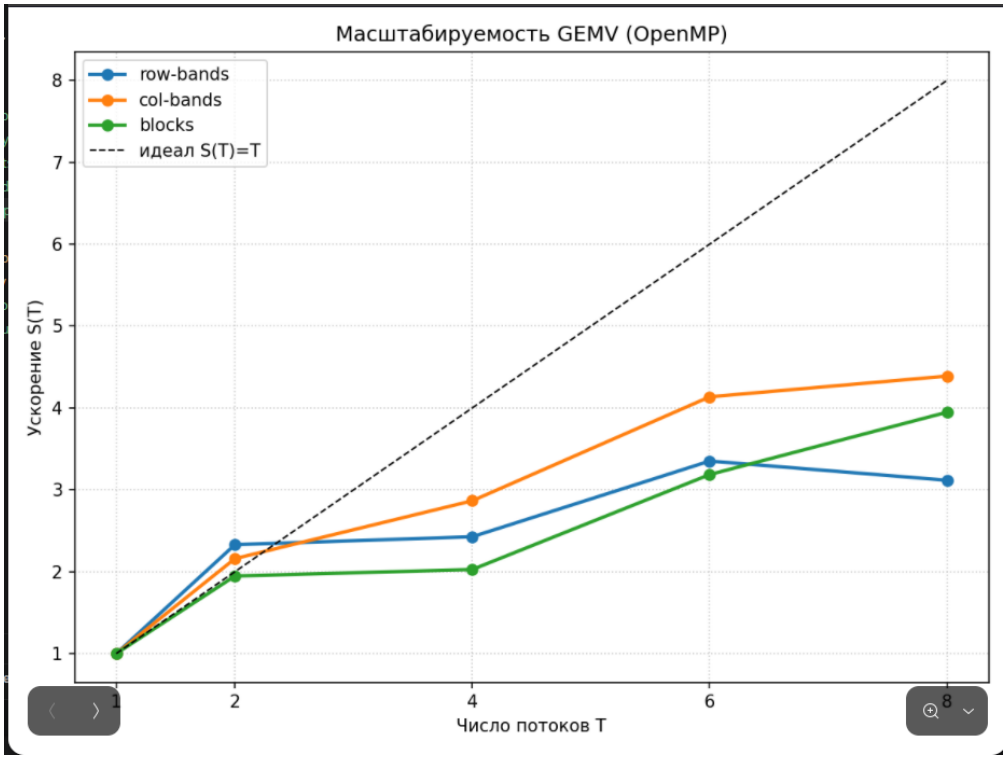


Рисунок 1 — Ускорение трёх схем распараллеливания при $T = 1,2,4,6,8$

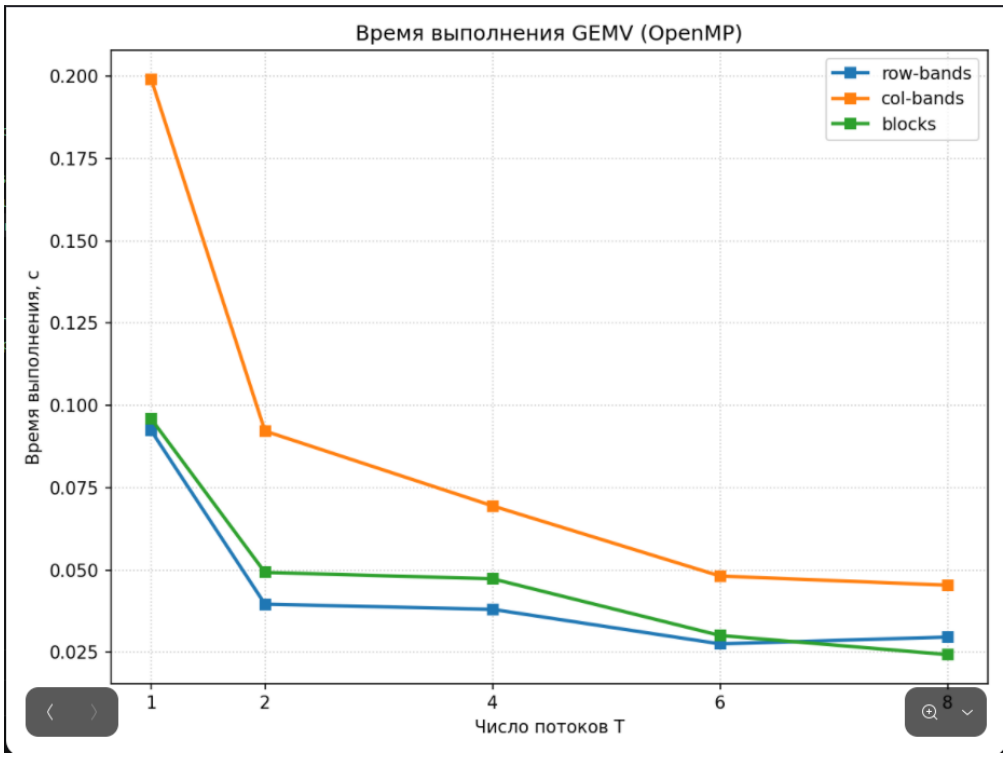


Рисунок 2 — Абсолютное время выполнения трёх схем

1.5 Анализ результатов

1. Все три схемы дают результат, совпадающий с последовательным эталоном с точностью 10^{-9}.
2. **Ленточное разбиение по строкам** — наилучшее ускорение. Причина — отсутствие гонок по данным: каждый поток пишет свою часть c, поэтому нужен только один барьер в конце `parallel for`, а дополнительных буферов не требуется. Дополнительно достигается лучшая пространственная локальность (матрица хранится по строкам, поток читает подряд идущие области).
3. Ленты по столбцам и блоки уступают из-за гонок по c_i: требуется $O(mT)$ памяти на локальные буферы и критическая секция при сведении.
4. Ускорение растёт почти линейно до $T = 4$ (число физических ядер). При $T = 6$ и $T = 8$ прирост замедляется по двум причинам: гипертрединг (логические ядра делят ALU и каналы памяти физического ядра) и memory-bound характер задачи (пропускная способность DRAM насыщается уже на 4 потоках).
5. Эффективность $E(T)$ падает с ростом T: при $T = 8$ у row-bands $E \approx 0.65$, у col/blocks ≈ 0.5. Это согласуется с оценкой $S(T) \leq B_{\text{peak}}/B_{\text{seq}}$.
6. t_{ref} во всех запусках практически одинаково (≈ 0.94 с) и не зависит от T, поскольку эталонный алгоритм последовательный.

1.6 Контрольные вопросы

1.6.1 Почему row-bands быстрее col-bands и blocks?

Разбиение по строкам даёт естественный параллелизм без гонок: выходные c_i не пересекаются между потоками. Col-bands и blocks вынуждены накапливать вклад в локальных буферах $O(mT)$ и сводить их через критическую секцию, что добавляет синхронизацию и давление на память. В memory-bound задаче это критично.

1.6.2 Зачем нужен `schedule(static)`?

Все ленты/блоки имеют одинаковую трудоёмкость, поэтому статическое распределение заранее делит итерации на равные порции. Это устраняет накладные расходы на динамическую раздачу работы и исключает голодание потоков в конце.

1.6.3 Что даёт `omp_get_wtime` по сравнению с `clock`?

`clock()` измеряет суммарное CPU-время всех потоков, что непригодно для оценки ускорения. `omp_get_wtime()` даёт реальное wall-clock время с высоким разрешением.

1.6.4 Почему при $T = 6, 8$ нет линейного ускорения?

Две причины: (1) гипертрединг — логические ядра делят ресурсы физического; (2) задача memory-bound — пропускная способность DRAM насыщается на 4 потоках, дальнейшее увеличение T не ускоряет чтение.

1.6.5 Почему `t_ref` не зависит от T?

Функция `matvec_ref` не содержит директив OpenMP и всегда выполняется в одном потоке. Значение `omp_set_num_threads(T)` на неё не влияет.

Вывод

В ходе работы реализованы три схемы параллельного умножения матрицы на вектор средствами OpenMP. Наилучшую масштабируемость показало ленточное разбиение по строкам, так как оно естественным образом даёт независимые единицы работы без гонок по данным. Показано, что для memory-bound задач (GEMV) ускорение ограничено сверху пропускной способностью памяти и накладными расходами синхронизации; при T, превышающем число физических ядер, эффективность резко снижается.